

C Programming

8_preprocessor

Директива #include

Заголовочный файл нужен только для:

Описания прототипов функций либо
объявления новых типов данных (структуры,
объединения)

```
#include <stdio.h>
```

Угловые скобки <> — говорит компилятору
искать в стандартных путях (обычно
/usr/include в linux).

```
#include "my_header.h"
```

"" — Искать в текущем каталоге

Опцией -I компилятору можно задать путь до
заголовочных файлов компилятору, чтобы в
include не прописывать полный путь до
заголовочного файла.

```
gcc prog.c -o prog -I ./include/
```

#define

#define – директива, предназначенная для макро определений.

С точки зрения хорошего стиля кода принято все макро определения пишут **большими буквами**.

#define используется обычно в двух случаях:

- для определения констант (#define N 5)
- для определения макросов (подобие функции, но не это не функция!)

```
#define K 123 // K определена как 123
```

```
// ..
```

```
#undef K // а теперь K больше не существует
```

```
#define MIN(a, b) ((a < b) ? a : b)
```

```
MIN(1, 2); // -> ((1 < 2) ? (1 : 2))
```

```
#define ADD(a, b) a + b
```

```
c = ADD(5, 3); // -> 5 + 3
```

```
#define ADD(a, b) a + b  
int a = 5, b = 3;  
int c = ADD(a, b); // c?  
int d = 4 * ADD(a, b); // d?
```

```
#define MIN(a, b) ((a < b) ? a : b)
```

```
MIN(func1(), func2());
```

Предопределенные константы

- `__LINE__` - номер строки
- `__FILE__` - имя файла
- `__FUNCTION__` - имя функции

```
printf("Error at LINE=%d, FILE=%s, function=%s\n",  
      __LINE__, __FILE__, __FUNCTION__);
```

```
printf("%s:%d\n", __FILE__, __LINE__); // CTRL+ЛКМ по этой ссылке в VSCode
```

- `__DATE__` - дата компиляции программы
- `__TIME__` - время компиляции программы

```
printf("Compilation time: date=%s, time=%s\n", __DATE__, __TIME__);
```

Условная компиляция

```
#if B == 5
// ..
#else
// ..
#endif
```

```
gcc src/cond.c -DB=5
```

```
#ifdef DEBUG
// ..
#endif
```

```
gcc src/cond.c -DDEBUG=1
```

```
#ifndef DEBUG
// ..
#else
// ..
#endif
```

Все эти директивы выполняются во время компиляции, а не во время работы программы!
Т.е. директивы не будут работать с внутренними переменными языка Си.

Полезно для добавления дебаг кода, для переносимости на различные архитектуры, для учитывания версии библиотек и т.д.

```
int sum(int x, int y) {  
    int sum = x + y;  
#ifdef DEBUG  
    // этот код будет скомпилирован только если DEBUG  
определен  
    printf("DEBUG: Value of x = %d\n", x);  
    printf("DEBUG: Value of y = %d\n", y);  
    printf("DEBUG: Sum calculated as %d\n", sum);  
#endif  
    return sum;  
}
```

```
#define PRINT_DEBUG(format, ...) \
do {
printf("%s:%d:%s(): " format, __FILE__, __LINE__, __FUNCTION__,
##__VA_ARGS__);
} while(0)
```

```
...
PRINT_DEBUG("Error: %s\n", message); // `;` точка с запятой
обязательно
```

```
...

do { if (1) printf("%s:%d:%s(): a !=%d\n", "cond.c", 5, "main()", a); } while(0);
```

Защита от повторного включения

```
// test.h

#ifndef _TEST_H_
#define _TEST_H_
// ..
int a; // определение
чего-либо
// ..
#endif // _TEST_H_
```

```
// test2.h

#include "test.h"
```

```
// test.c

#include "test.h"
#include "test2.h"

// ..
```

Github

<https://github.com/kruffka/C-Programming>

